Agentic AI Study For DDSI Lab 1st Sharing: GitHub & Langgrapgh & Context Engineering



Dept. of Industrial Engineering | Data-Driven Service Inovation Lab

2026. 10. 16. 발제자: 이수인.

Purpose & Flow of This Study

- 본 스터디는 하기 3가지 목적으로 진행해왔습니다.
- 앞으로 참여인원이 0명이 될 때까지 + 여력이 될 때까지 주욱 진행할 예정입니다.
- 제가 주도하고 있고, 우명균, 이진수, 최우진, 김지호 연구원이 팔로우업해주고 있습니다.

1. GitHub 협업 방식을 익히고 익숙해지자

0주차: Awesome TM Repo Contributor가 되기

↓

1주차: GitHub 협업방식 복습

→

2. Agentic AI 개발에 쓰이는 코드에 익숙해지자

1주차: LangGraph 기본

↓

2주차: LangGraph _ Graph API

→

3. Agentic AI 개발 지식을 쌓고 트렌드를 팔로우 업 하자

2주차: Context Engineering

↓

3주차: 1019(월) 17시 진행 예정(프워룸)

오늘의 공유 세션 흐름

- 오늘 공유 세션 시간은 아래의 흐름에 따라 진행됩니다.
- 다만 내용이 지금껏 진행해왔던 내용이 꽤나 방대하므로 모두 간단하게 핵심만 설명할 예정입니다.

1. GitHub

1-1. 깃헙 협업 Flow

1-2. Awesome TM 프로젝트 소개

2. LangGraph

2-1. LangGraph 기본

2-2. Graph API

3. Agentic AI 지식

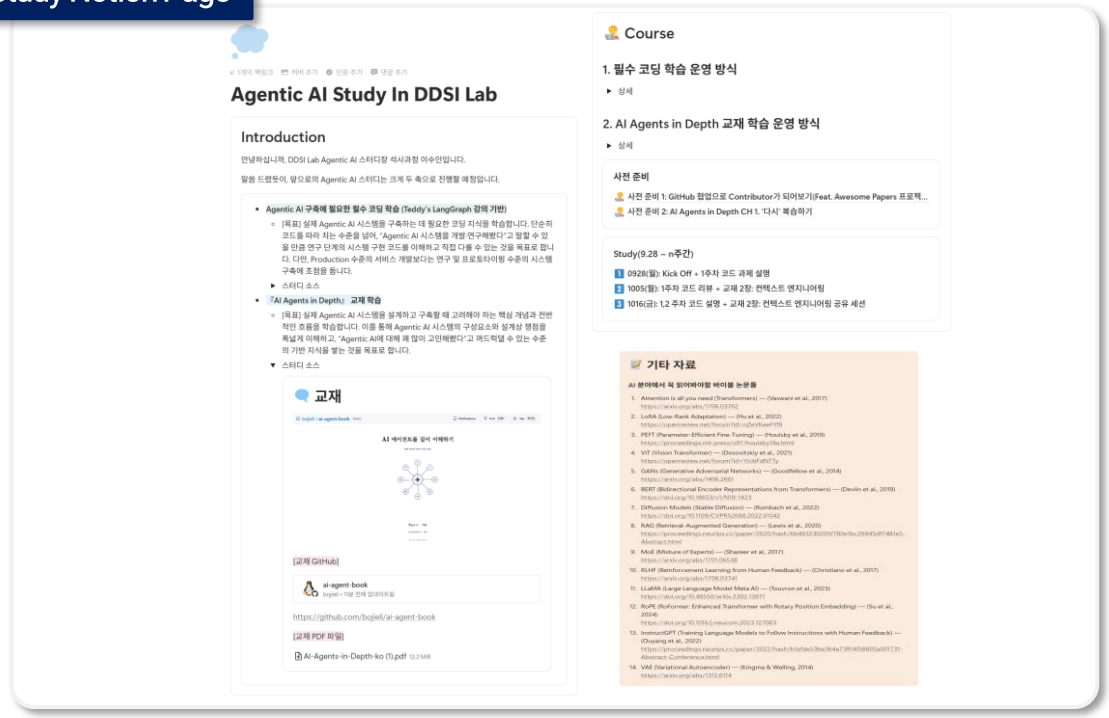
3-1. Agentic AI란 무엇인가(복습)

3-2. Context Engineering

스터디 자료

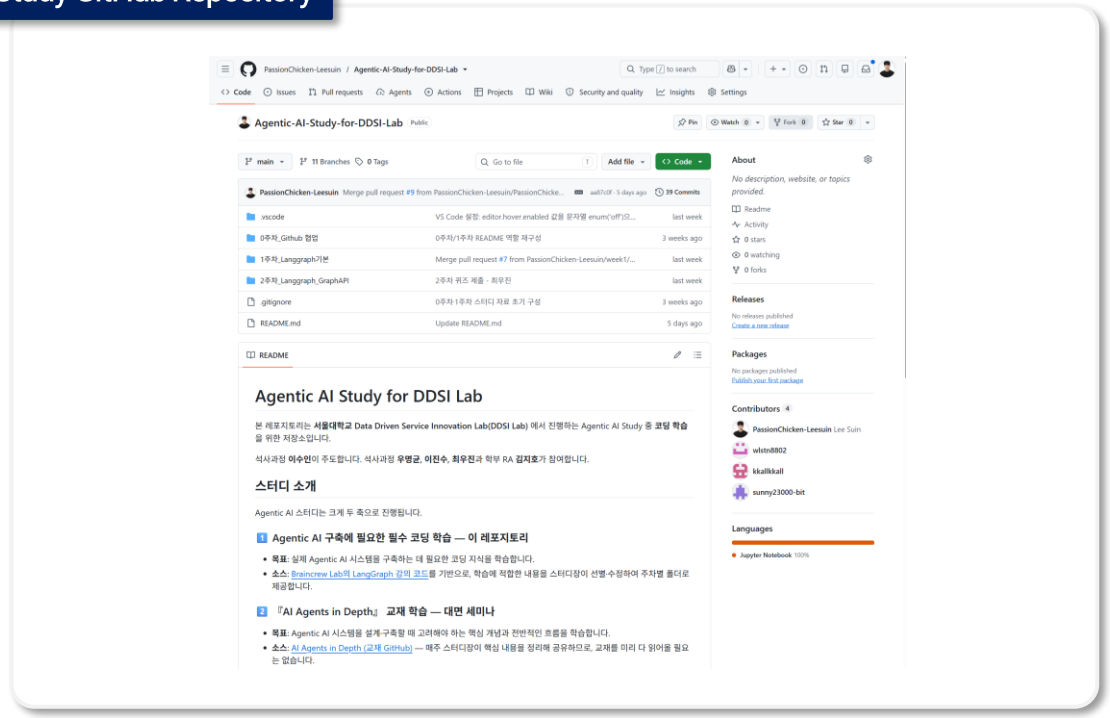
- 관심이 생기셔서 더 깊게 공부하고 싶으신 분은 스터디 이후 따로 말씀주시거나,
- 하단의 Notion Page와 GitHub Repository Link를 참고해주십시오.

Study Notion Page



Notion Link: https://app.notion.com/p/Agentic-AI-Study-In-DDSI-Lab-3c631bc3444c804b8679d87a99cb30c2?source=copy_link

Study GitHub Repository



GitHub Link: https://github.com/PassionChicken-Leesuin/Agentic-AI-Study-for-DDSI-Lab

1. GitHub

1-1. 깃헙 협업 Flow

1-2. Awesome TM 프로젝트 소개

01

1-1. 깃헙 협업 Flow

GitHub 저장소에는 `브랜치(Branch)`라는 개념이 있습니다. 브랜치는 하나의 프로젝트에서 여러 사람이 동시에 작업할 수 있도록 작업 공간을 나누는 '가지'라고 생각하시면 됩니다.

우리 레포지토리에서 `main 브랜치`는 검토가 완료된 공식 결과물이 모이는 기본 브랜치입니다. 연구원분들은 main을 직접 수정하지 않고, 각자 이름의 개인 작업 브랜치를 만들어 그 안에서 작업합니다. 작업이 완료되면 "이 변경 내용을 main 브랜치에 반영해도 될까요?"라고 요청하게 되는데, 이를 `Pull Request(PR)`라고 합니다.

협업 작업은 먼저 GitHub에 있는 최신 main 브랜치의 내용을 내 컴퓨터로 가져오는 것부터 시작합니다. 이 과정을 `git pull`이라고 합니다. 다른 연구원이 먼저 반영한 변경 사항까지 받아온 뒤 작업해야, 서로 다른 버전의 파일을 기준으로 작업하는 상황을 줄일 수 있습니다.

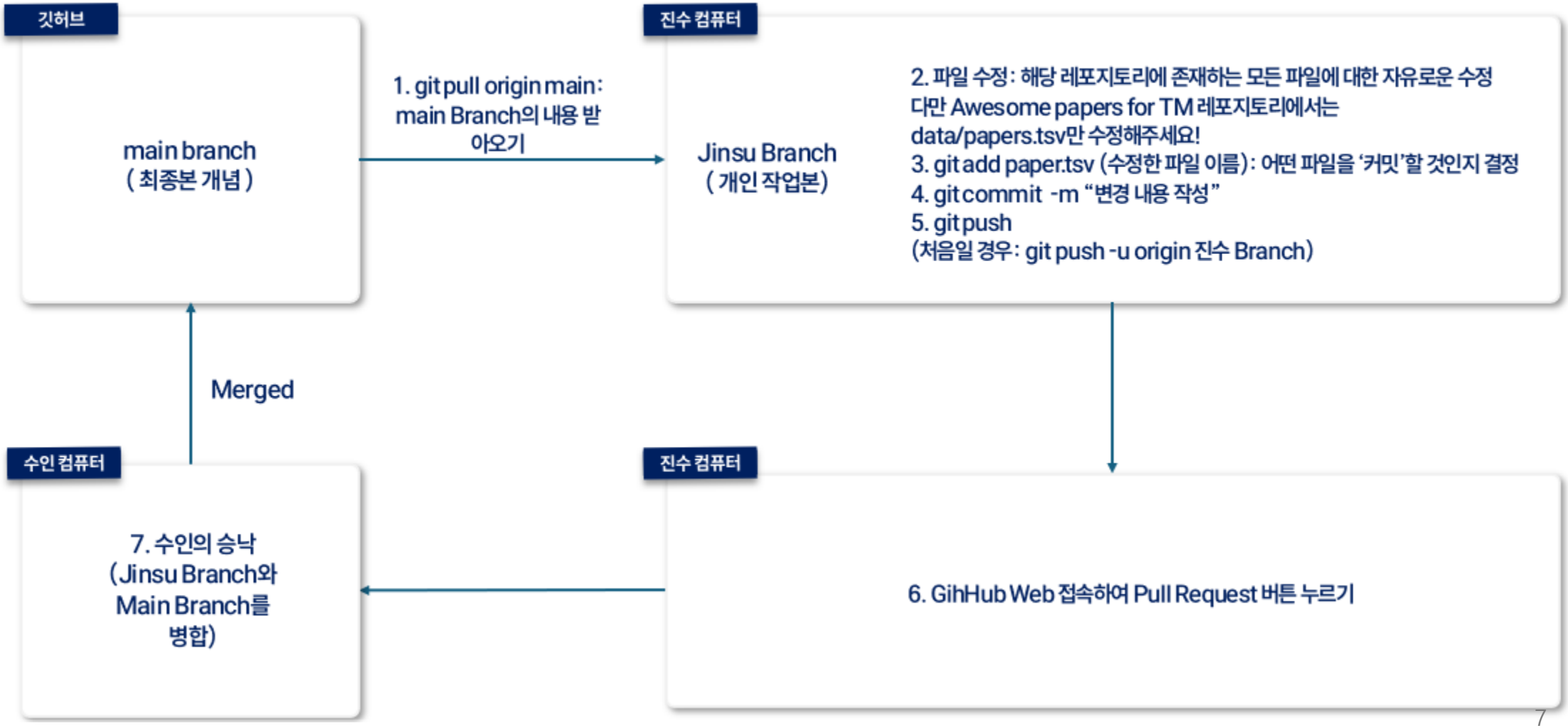
그다음 개인 작업 브랜치에서 파일을 수정하고, 아래 흐름으로 변경 내용을 GitHub에 올린 뒤 Pull Request를 생성합니다.

`git pull` → `VS Code에서 파일 수정` → `git add` → `git commit` → `git push` → `Pull Request(PR)`

- `git pull` : GitHub에 있는 최신 변경 내용을 내 컴퓨터로 가져와 반영합니다. 작업을 시작하기 전에 실행합니다.
- `VS Code에서 파일 수정` : 개인 작업 브랜치에서 필요한 파일을 수정합니다.
- `git add` : 수정한 파일 중 이번 변경에 포함할 파일을 선택합니다.
- `git commit` : 선택한 변경 내용을 내 컴퓨터의 Git 기록에 저장하고, 변경 내용을 설명하는 메시지를 남깁니다.
- `git push` : 내 컴퓨터에 저장한 커밋을 GitHub의 개인 작업 브랜치에 업로드합니다.
- `Pull Request(PR)` : GitHub에 업로드된 개인 브랜치의 변경 내용을 검토해 달라고 요청하고, 이를 main 브랜치에 병합해 달라고 제안합니다.

1-1. 깃헙 협업 Flow

이를 정리한 전체 협업 흐름은 아래 그림과 같습니다. 처음에 저장소를 내 컴퓨터로 내려받는 `git clone`을 실행하고, 개인 작업 브랜치를 생성하는 코드는 뒤에서 설명드리겠고, 먼저 아래의 협업 흐름을 이해해 주시면 좋겠습니다.



1-1. 깃헙 협업 Flow

1. 진수로 예를 들겠습니다. 먼저 진수는 자신의 컴퓨터에서 최신 `main` 브랜치 내용을 가져옵니다. 이때 다른 사람이 이미 반영한 최신 변경 사항을 받아온 뒤, 그 상태를 기준으로 개인 작업 브랜치를 만듭니다.

```
# main 브랜치로 이동한 후, main branch의 내용을 받아오는 명령어
git switch main
git pull origin main

# 진수만의 branch를 하나 만들고(이름: jinsu-branch), jinsu 브랜치로 이동
git branch jinsu-branch
git switch jinsu-branch
```

2. 이후 진수는 자신의 개인 브랜치에서 필요한 파일을 수정합니다.

   [Example]

   우리가 협업하는 Awesome papers for tm 저장소에서는 논문을 추가하거나 수정할 때는 원본 데이터인 `data/papers.tsv`를 VS코드 상에서 수정하고, (python scripts/build_readme.py 명령어)로 README를 다시 생성합니다(2절에서 더 자세히 설명드리겠습니다.)

3. 수정이 끝나면, Git에게 "이 파일들을 이번 변경 사항에 포함하겠다"고 알려줍니다. 이 과정이 `git add` 입니다.(위의 예에서는 papers.tsv를 수정했고, 파이썬 명령어를 실행하여 결과적으로 README.md도 수정했으므로 두가지 파일을 add 합니다.)

```
git add data/papers.tsv README.md
```

4. 다음으로 변경 내용을 내 컴퓨터의 Git 기록에 저장합니다. 이 과정이 `git commit` 이며, 어떤 작업을 했는지 짧은 메시지로 남깁니다.

```
git commit -m "Add a new paper"
```

5. 이제 내 컴퓨터에만 저장되어 있던 작업 내용을 GitHub의 개인 브랜치로 업로드합니다. 처음 업로드할 때는 아래와 같이 입력합니다.

```
git push -u origin jinsu-brange
```

   (이후 같은 브랜치에 추가로 작업할 때는 `git push`만 입력하면 됩니다. `-u` 명령어의 경우 처음에 로컬과 원격 저장소를 연결하기 위해 사용하는 것입니다.)

6. 업로드가 끝나면 GitHub 웹페이지에서 Pull Request를 생성합니다. 이는 "제 개인 브랜치에서 작업한 내용을 `main`에 반영해 주세요"라고 요청하는 단계입니다.
7. 마지막으로 이수인(레포지토리의 소유자겄죠!)이 Pull Request의 변경 내용을 확인한 뒤, 문제가 없으면 병합(Merge)합니다. 그러면 진수의 개인 브랜치에서 작업한 내용이 공식 브랜치인 `main`에 반영됩니다.

즉, 개인 브랜치에서 안전하게 작업 → GitHub에 업로드 → Pull Request로 검토 요청 → 검토 후 main에 병합이 전체 협업 흐름입니다.

1-2. Awesome TM 프로젝트 소개

- GitHub에는 특정 주제의 논문·도구·자료를 지속적으로 수집하고 정리하는 Awesome Repository가 매우 많이 존재합니다.
- 특히 빠르게 발전하는 LLM 관련 연구를 매주 추적하고 공유하기 위한 목적으로, 다양한 분야에서 이러한 레포지토리가 만들어지고 활발히 활용되고 있습니다.

I. awesome-ai-agent-papers — agent engineering, memory, evaluation, workflows 관련 논문 목록 (깃헙 스타 1,700개)
II. Awesome-LLM-Agents-Scientific-Discovery — biomedical research agents (깃헙 스타 80개)

- 그러나 기술경영 분야에서는 Agentic AI와 LLM의 활용 가능성에 관한 연구가 빠르게 늘어나고 있는 현 상황에도, 관련 Awesome Repository는 전무한 실정이었습니다.
- 연구원분들과 함께 논문을 추가하고, 이미 등록된 논문을 검토하며, 필요하다면 수정·삭제하는 과정을 통해 신뢰할 수 있는 문헌 목록을 만들어 가고자 시작한 프로젝트입니다.
- 레포지토리가 꾸준히 관리되고 많은 연구자에게 활용될수록, 우리 연구실이 기술경영 분야의 LLM·Agentic AI 연구를 지속적으로 탐색하고 정리해 왔다는 흔적도 남게 됩니다.
- 깃헙 협업 Flow를 익힐 겸 해당 프로젝트에 참여하고 싶으신 분들께서는 스터디 이후 말씀 주십시오.

이 프로젝트의 목적은 다음과 같습니다.

1. 기술경영 분야에서 LLM Application을 연구하는 모든 연구자가 참고할 수 있는, 신뢰도 높은 문헌 목록을 만든다.
2. 레포지토리에 등록된 논문을 함께 읽고 검토하며, 새롭게 발표되는 관련 연구를 빠르게 추적하고 반영한다.
3. 논문 목록을 함께 관리하는 과정에서 GitHub 기반의 협업 방식을 경험하고 익힌다.

1-2. Awesome TM 프로젝트 소개

이 프로젝트에서의 협업 방식

이 프로젝트에서는 레포지토리에 등록된 논문 목록을 검토하고, 논문을 추가하거나 삭제하는 작업을 진행합니다.

논문을 직접 읽어본 뒤 해당 논문이 기술경영 분야의 LLM Application 연구와 관련성이 낮다고 판단되거나, 목록에 포함할 만한 가치가 부족하다고 판단되면 삭제를 제안해 주시면 됩니다. 반대로 중요한 논문인데 목록에 빠져 있다고 판단되면 새로 추가해 주시면 됩니다.

앞선 1절에서 GitHub 협업의 기본 흐름을 설명드렸으므로, 해당 내용을 이해하고 계신다는 전제하에 이 프로젝트의 구체적인 작업 방식을 안내하겠습니다.

1. **main 브랜치에서는 직접 작업하지 않습니다.**

   main 브랜치에는 보호 규칙이 설정되어 있어 직접 수정할 수 없도록 되어 있습니다. 작업을 시작할 때마다 최신 main 브랜치를 기준으로 새 작업 브랜치를 생성하거나, 아직 병합되지 않은 기존 작업을 이어가는 경우에는 해당 개인 브랜치로 이동하여 작업해 주세요.

2. **수정할 원본 파일은 `data/papers.tsv` 입니다.**

   논문을 추가하거나 삭제할 때는 아래 파일만 수정합니다.

   ```
   data/papers.tsv
   ```

   이 파일이 레포지토리의 논문 목록을 관리하는 원본 데이터입니다.

3. **수정 후에는 README 생성 명령어를 반드시 실행합니다.**

   `data/papers.tsv` 를 수정한 뒤에는 터미널에서 아래 명령어를 실행해 주세요.

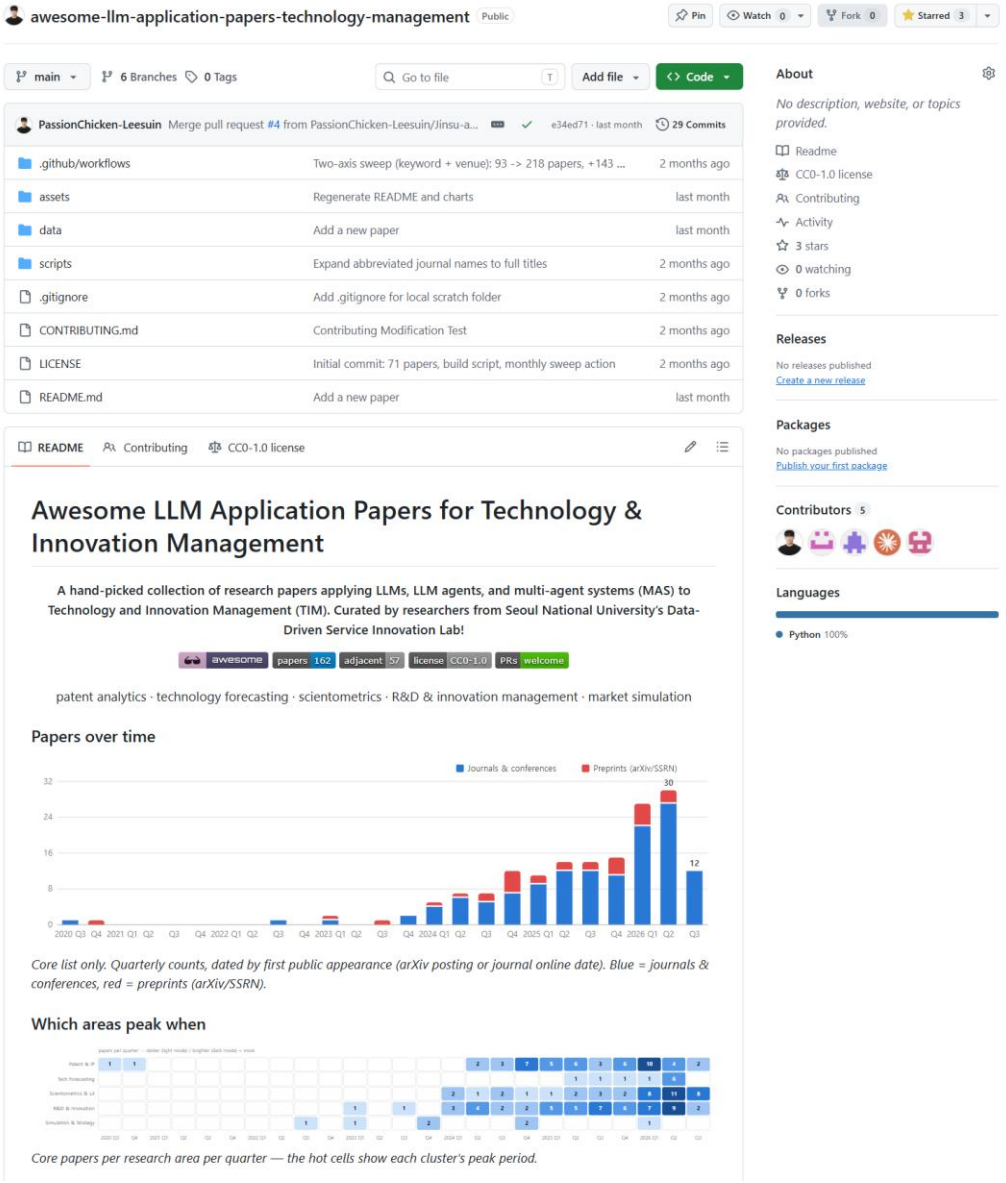
   ```
   python scripts/build_readme.py
   ```

   이 명령어를 실행해야 `README.md` 의 논문 목록, 논문 수 배지, 추세 그래프, 학회·저널별 표 등 자동 생성되는 내용이 함께 최신 상태로 갱신됩니다.

4. **`data/papers.tsv` 와 자동 생성된 `README.md` 를 함께 커밋합니다.**

   ```
   git add data/papers.tsv README.md
   ```

   이후에는 앞서 설명한 순서대로 커밋, 푸시, Pull Request 생성을 진행해 주세요.



2. Langrapgh

2-1. Langgraph 기본

2-2. Graph API

02

2-1. LangGraph 기본(1) 1주차 _ Langgraph기본/notebooks/01-LangGraph-Introduction.ipynb

TypedDict

- TypedDict는 LangGraph에서 State를 선언할 때 가장 자주 사용되는 방식입니다.
- 일반 Dict(파이썬 딕셔너리)와 달리 타입 검사를 제공하기 때문에, 코드 작성시 IDE나 타입 체커가 오류를 미리 잡아낼 수 있습니다.
- dict는 키와 값의 타입을 일반적으로 지정하는 반면 (예: Dict[str, str]), TypedDict는 각 키에 대해 구체적인 타입을 지정할 수 있습니다(가독성 굿)

```python
# TypedDict와 Dict의 차이점 예시
from typing import Dict, TypedDict

# 일반적인 파이썬 딕셔너리(dict) 사용
sample_dict: Dict[str, str] = {
    "name": "테디",
    "age": "30",  # 문자열로 저장 (dict 에서는 가능)
    "job": "개발자",
}


# TypedDict 사용
class Person(TypedDict):
    name: str
    age: int  # 정수형으로 명시
    job: str


typed_dict: Person = {"name": "셜리", "age": 25, "job": "디자이너"}
```

Annotated

- 'Annotated'는 Python의 'typing' 모듈에서 제공하는 특별한 타입 힌트로, 기존 타입에 메타데이터를 추가할 수 있게 해줍니다. LangGraph에서는 상태(State) 필드에 리듀서(reducer)함수를 지정할 때 주로 사용됩니다.
  'Annotated'를 사용하면 타입 힌트에 추가 정보를 포함시킬 수 있어 코드의 가독성을 높이고, 더 자세한 타입 정보를 제공할 수 있습니다.
- 아래는 가장 기본적인 'Annotated' 사용 방법입니다. 타입 힌트와 함께 문자열 형태의 메타데이터를 추가하여 변수의 의미를 명확히 할 수 있습니다.

```python
from typing import Annotated

# 타입 힌트에 메타데이터(설명)를 추가하는 기본 사용법
name: Annotated[str, "사용자 이름"]
age: Annotated[int, "사용자 나이 (0-150)"]
```

Reducer

- LangGraph에서 'Annotated'는 상태(State) 필드에 리듀서(reducer) 함수를 지정할 때 사용됩니다. 리듀서 함수는 상태가 업데이트될 때 기존 값과 새 값을 어떻게 결합할지 정의합니다.
- 'add_messages'는 LangGraph에서 가장 많이 사용되는 리듀서로, 새로운 메시지를 기존 메시지 리스트에 추가하는 역할을 합니다. 이를 통해 대화 기록을 자연스럽게 누적할 수 있습니다.
- 아래 코드에서는 'messages' 필드에 'add_messages' 리듀서를 적용하여 메시지가 자동으로 누적되도록 설정합니다.

```python
from typing import Annotated, TypedDict
from langgraph.graph import add_messages


# add_messages 리듀서를 State 필드에 적용하는 예시
class MyData(TypedDict):
    messages: Annotated[list, add_messages]
```

2-1. LangGraph 기본(2): 1주차_Langgraph기본/notebooks/01-LangGraph-Models.ipynb

Model 호출

- 이후 LangChain으로 LLM을 다루는 기본 방법을 배웠습니다.
- 모델은 init_chat_model("openai:gpt-5.4-mini")처럼 provider:model 문자열 하나로 간단히 만들 수 있고,
- ChatOpenAI 같은 클래스를 직접 쓰면 temperature, max_tokens, timeout, reasoning_effort 같은 옵션을 더 세밀하게 정할 수 있습니다.
- 모델을 호출하는 방법은 네 가지입니다.
    1. 응답이 다 만들어질 때까지 기다렸다가 한 번에 받는 invoke()
    2. 토큰이 생성되는 대로 조각(AIMessageChunk)으로 받는 stream()
    3. 이 둘을 비동기로 쓰는 ainvoke()/astream()
    4. 여러 요청을 병렬로 한꺼번에 처리하는 batch().
- 응답은 AIMessage 객체로 오는데, 답변 텍스트는 content에, 토큰 사용량은 usage_metadata에, 모델명이나 종료 사유 같은 제공자 정보는 response_metadata에 들어 있습니다.

기능 & 운영

- 기능 면에서는 세 가지를 익혔습니다.
    1. bind_tools()로 Pydantic 도구를 연결하면 모델이 직접 답하는 대신 tool_calls에 호출할 도구 이름과 인자를 담아 돌려줍니다(이때 content는 빈 문자열입니다).
    2. with_structured_output()을 쓰면 응답이 정해 둔 스키마를 따르게 할 수 있습니다.
    3. MultiModal로 이미지를 넣어 분석할 수 있습니다.
- 운영에 필요한 것으로는
    1. UsageMetadataCallbackHandler로 여러 번 호출한 토큰 사용량을 모델별로 합산하는 법,
    2. config에 run_name·tags·metadata를 넣어 LangSmith에서 실행을 추적하는 법,
    3. logprobs로 모델이 답에 얼마나 확신하는지 확인하는 법도 다뤘습니다.

2-1. LangGraph 기본(3): 1주차_Langgraph기본/notebooks/01-QuickStart-LangGraph-Tutorial.ipynb

Chatbot 구축 실습

이후 LangGraph로 챗봇을 처음부터 만들고 기능을 하나씩 붙여 가는 과정을 실습했습니다.

1. 먼저 messages: Annotated[list, add_messages] 형태로 State를 정의합니다.
2. add_messages 리듀서 덕분에 새 메시지가 기존 목록을 덮어쓰지 않고 뒤에 쌓입니다.
3. 그다음 StateGraph(State)에 add_node()로 LLM을 호출하는 노드를 추가하고, add_edge()로 START → chatbot → END 경로를 연결한 뒤, compile()로 실행 가능한 그래프를 만듭니다.
4. 도구를 붙일 때는 bind_tools()로 LLM에 도구를 연결하고, 도구를 실제로 실행하는 ToolNode를 추가합니다.
5. 이어서 add_conditional_edges("chatbot", tools_condition)을 넣으면 마지막 AI 메시지에 tool_calls가 있을 때는 tools 노드로, 없을 때는 END로 갈림길이 생깁니다.
6. 여기에 tools → chatbot 엣지를 더하면 chatbot과 tools를 오가는 순환 구조가 완성됩니다.

Memory & HITL

메모리에 대한 개념과 Human In the Loop 구축도 실습했습니다.

1. compile(checkpointer=InMemorySaver())로 체크포인터를 연결하면 같은 thread_id 안에서 대화 상태가 저장되어 이어지고,
2. InMemoryStore를 쓰면 user_id 기준의 장기 기억을 만들 수 있어 스레드가 달라도 사용자 정보가 유지됩니다.
3. 저장된 상태는 get_state()로 확인합니다.
4. Human-in-the-Loop에서는 도구 안에서 interrupt()를 호출해 그래프를 일시정지하고,
5. 사람의 응답을 Command(resume=...)에 담아 넘기면 멈췄던 지점부터 다시 실행됩니다.
6. 마지막으로 State에 human_feedback 같은 필드를 추가해 상태를 커스터마이징하고,
7. 도구가 문자열 대신 Command(update=...)를 반환해 상태를 직접 고치는 방법도 익혔습니다.

2-2. GraphAPI (1): 2주차 _ Langgraph _ GraphAPI/notebooks/02-LangGraph-Messages.ipynb

Messages

- LangChain에서 모델과 대화하는 기본 단위인 메시지를 배웠습니다. 메시지는 역할(role), 콘텐츠(content), 메타데이터로 이루어지고, 핵심 유형은 네 가지입니다.
  1. 모델의 역할과 행동 지침을 정하는 SystemMessage,
  2. 사용자 입력인 HumanMessage,
  3. 모델의 응답이자 tool _ calls를 담는 AIMessage,
  4. 도구 실행 결과를 모델에 돌려주는 ToolMessage입니다.
- 모델은 문자열 하나로도 호출할 수 있고(내부에서 HumanMessage로 바뀝니다), 여러 턴의 대화는 메시지 객체 리스트로 넘깁니다.
- 같은 대화를 {"role": "system"/"user"/"assistant", "content": ...} 형태의 딕셔너리로 써도 됩니다.
- 메시지에는 name이나 id 같은 메타데이터를 붙여 발신자를 구분하고 메시지를 추적할 수 있습니다.

Tools

- 도구 호출 흐름은 HumanMessage → tool _ calls가 담긴 AIMessage → ToolMessage 순서로
- 메시지를 쌓은 뒤 모델에 넘기면, 모델이 도구 결과를 해석해 최종 답을 만듭니다.
- 이때 ToolMessage의 tool _ call _ id는 해당 AIMessage에 있는 tool _ calls의 id와 같아야 합니다.
- 이미지 같은 멀티모달 콘텐츠를 보내는 방법은 두 가지입니다.
  1. 하나는 content=[...]에 제공사(예: OpenAI)의 네이티브 형식을 그대로 넣는 방법인데, 모델을 바꾸면 메시지 코드도 고쳐야 합니다.
  2. 다른 하나는 content _ blocks=[...]에 LangChain 표준 블록({"type": "image", "url": ...})을 쓰는 방법으로, 모델을 바꿔도 코드가 그대로 동작하고 네이티브 형식 메시지도 .content _ blocks로 읽으면 표준 형태로 바뀌어 나옵니다.
- 그래서 여러 모델을 오가는 에이전트 시스템에서는 표준 블록을 기본으로 쓰는 편이 안전합니다.

🧩 1주차 퀴즈 ① — State

이 노트북에서 배운 내용을 직접 실습해보는 퀴즈입니다. 빈칸(`_____`)을 채우고 셀을 실행해서 **"통과! 🎉"** 가 출력되면 성공입니다.

제출 방법

1. 퀴즈는 `submissions/본인이름/` 폴더에 미리 들어 있는 **본인 이름이 붙은 노트북**에서 풉니다. (`notebooks/` 원본은 수정 금지!)
2. 각 퀴즈의 빈칸을 채우고 실행 결과를 확인합니다.
3. `week1/본인이름` 브랜치에서 커밋 후 Pull Request를 올립니다. (자세한 방법: 1주차 README의 '학습 및 퀴즈 제출 방법')

Q1. TypedDict 정의하기

스터디 멤버를 표현하는 `StudyMember`를 `TypedDict`로 정의해보세요. 필드는 `name`(문자열), `week`(정수), `submitted`(불리언) 세 개입니다.

```python
from typing import TypedDict


class StudyMember(_____):
    name: _____
    week: _____
    submitted: _____


member: StudyMember = {"name": "홍길동", "week": 1, "submitted": True}
print(member)
print("통과! 🎉")
```

Python

Q2. Annotated로 리듀서 지정하기

LangGraph의 State에서 `messages` 필드에 `add_messages` 리듀서를 적용해보세요. (힌트: 이 노트북의 `MyData` 예시를 떠올려 보세요)

```python
from typing import Annotated, TypedDict
from langgraph.graph import add_messages


class ChatState(TypedDict):
    messages: _____[list, _____]

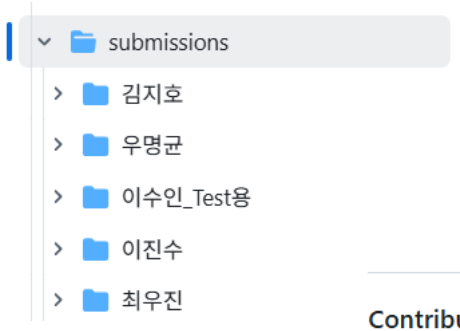

# 자동 채점: add_messages가 메타데이터로 붙어 있는지 확인
assert ChatState.__annotations__["messages"].__metadata__[0] is add_messages
print("통과! 🎉")
```

Python

Windows 정품 인증

GitHub Pull Request





Contributors 4


PassionChicken-Leesuin Lee Suin


wlstn8802


kkallkkall


sunny23000-bit

3. Agentic AI 지식

3-1. Agentic AI란 무엇인가

3-2. Context Engineering

03

3-1. Agentic AI란 무엇인가(1)

LLM capability comes from pretrained world knowledge and language ability, plus decision strategies learned through post-training.



Context includes more than the text input:
it covers the environment, user memory, domain knowledge, the agent's state, and task progress. And RAG

Tools include more than callable APIs: predefined tool calls, on-demand skills, code generation, delegating to sub-agents, asking users for help, and responding to external events.

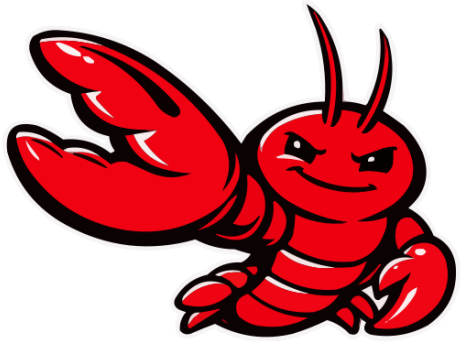
3-1. Agentic AI란 무엇인가(2)

Agent = Context + LLM + Tool

manus

Manus: Integrating previously separate spaces.

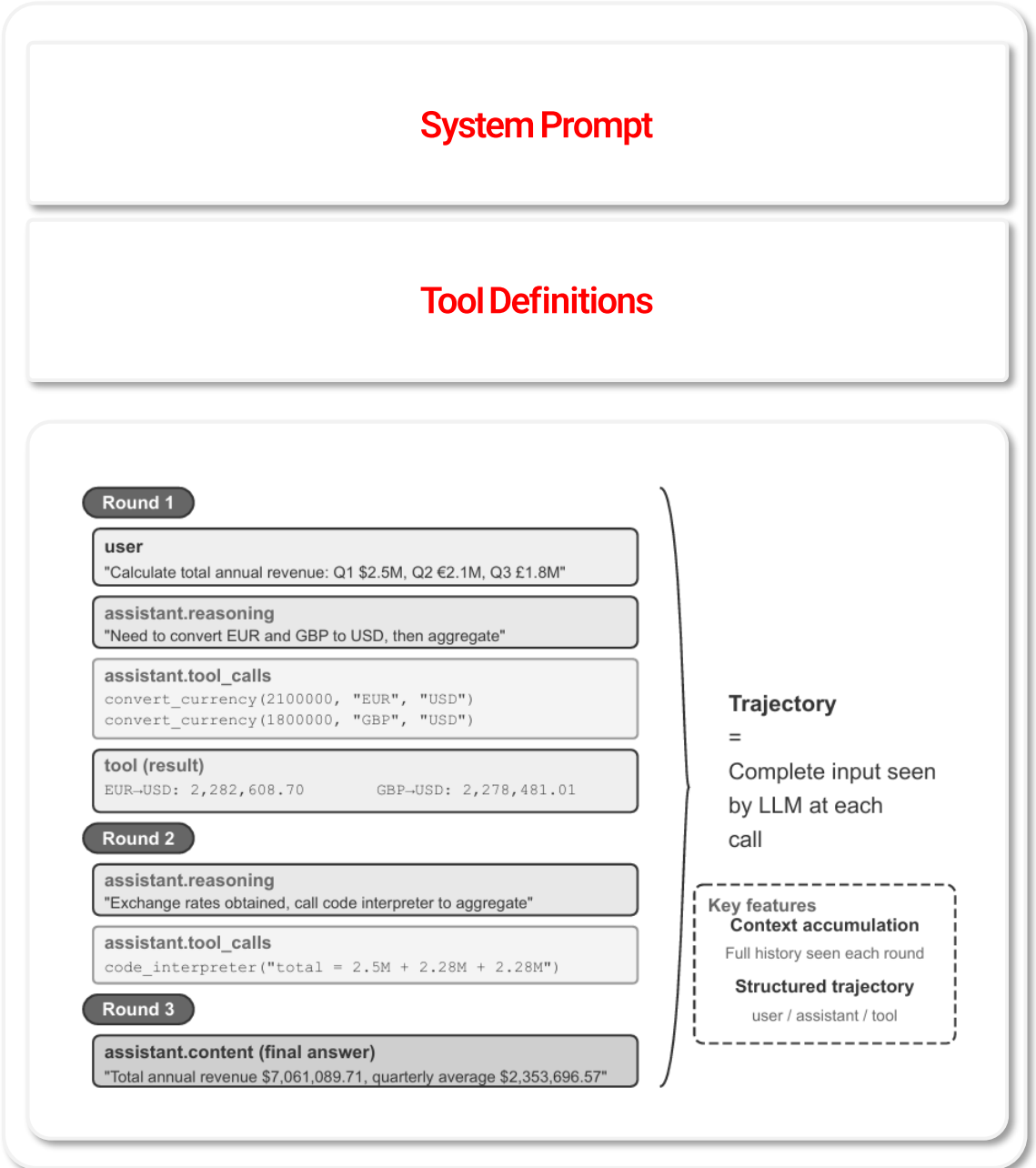
Before Manus, commercial agents were usually divided into deep research, coding, and computer-use systems. Manus combined their observation and action spaces into one system, enabling a single agent to work across product boundaries.



OpenClaw: Extending the interface into users' digital lives.

OpenClaw lets users interact with agents through everyday messaging apps and access both cloud and local files. This expands the agent's observation space and makes a wider range of personal data available for tasks.

3-1. Agentic AI란 무엇인가(3)



Fixed Part:
Can't be Modified After Designing the Agent

Cumulative Part

3-2. Context Engineering(1): Context Window

- 컨텍스트는 AI와 상호작용할 때 AI가 실제로 "보는" 모든 정보임.
- 대화기록 뿐 아니라 개발자가 미리 작성한 행동규칙(시스템지시), AI가 사용할 수있는외부기능의설명(도구설명), 그 밖의 정보도 포함
- 컨텍스트 창의 기본적인 구성 개요는 왼쪽 하단과 같음(교재 그림). Claude Code에서 타이핑 전에 기본적으로 구성되는 컨텍스트 창은 우측하단과 같음. (들어가서 시뮬레이션 돌려보시기를 권장!)

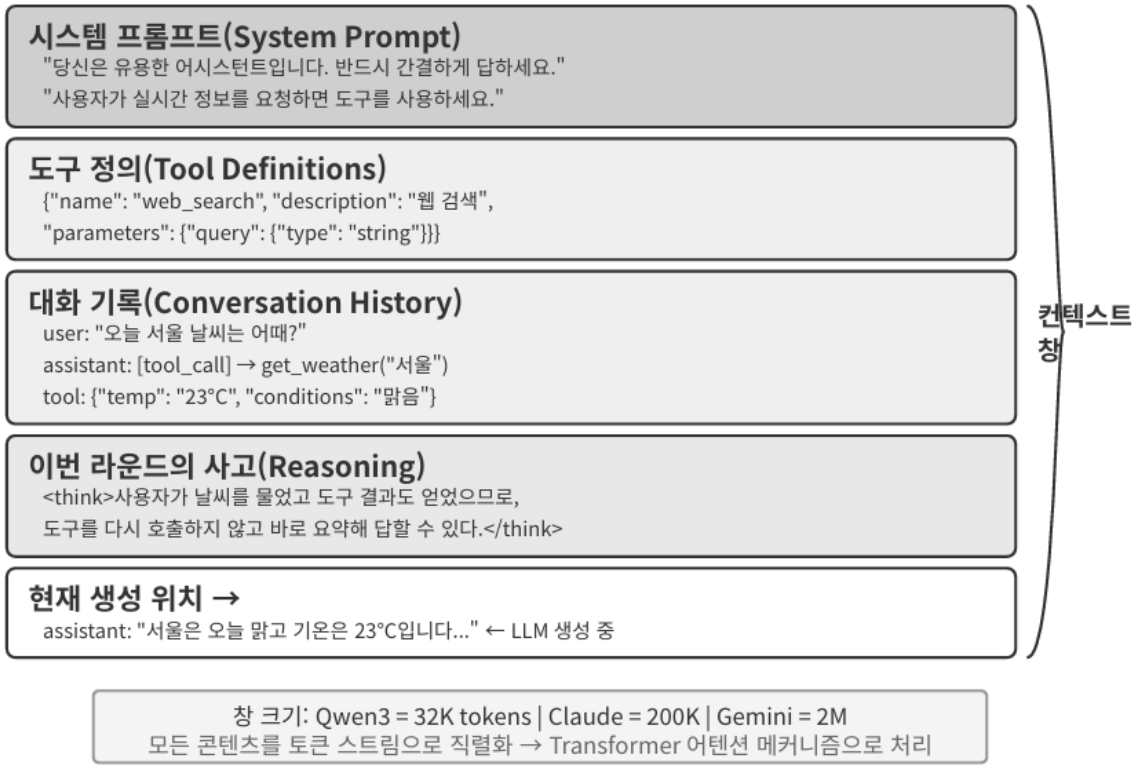
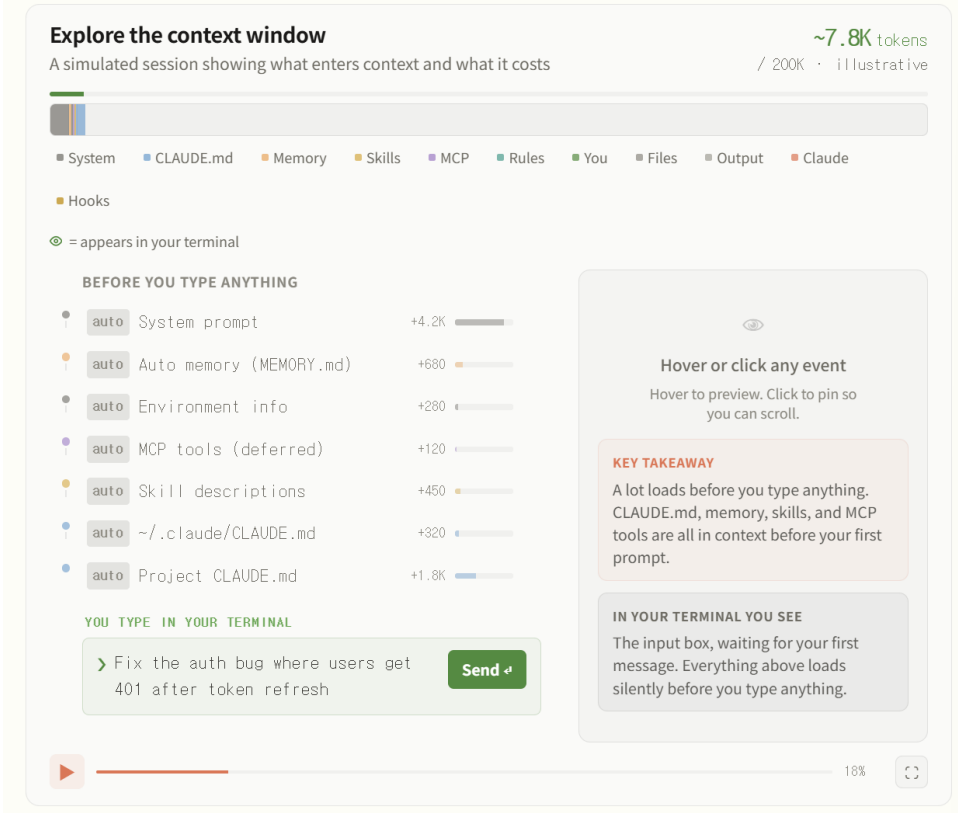


그림 2-1 컨텍스트 창의 구성 개요

교재 41pg 中



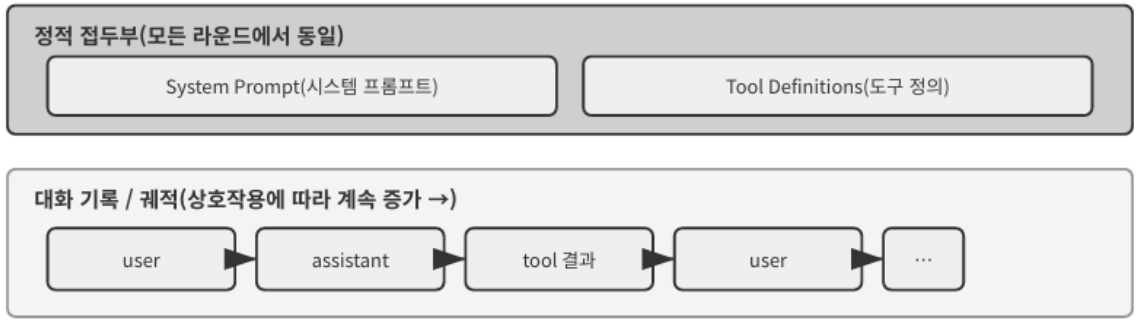
https://code.claude.com/docs/en/context-window

3-2. Context Engineering(1): Context Window

- 컨텍스트는 창에서도 위쪽 부분(시스템 프롬프트 +도구정의)은대화내내바뀌지않고,아래 쪽 부분(대화기록)은 상호 작용할 때 마다 늘어남.
- 이"정적 접두부 +궤적"구조는 뒤에서 다룰 KVCache 최적화와 컨텍스트 압축 같은 기법의 토대임. 접두부는 안정적으로 유지하고, 뒤쪽 궤적은 상충관계를 고려해 압축하거나 교체할 수 있음

2.2.5 API 관점에서 본 컨텍스트의 구성

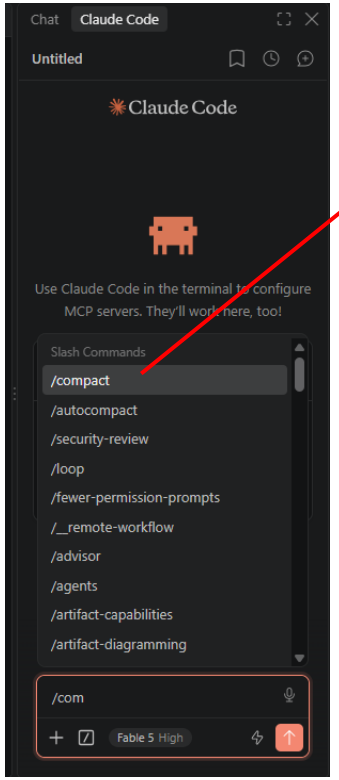
앞의 예시는 에이전트가 모델을 호출할 때마다 컨텍스트가 어떻게 완성되는지 보여 줍니다.



"정적 접두부 + 궤적" 구조: 접두부는 고정(KV Cache에 유리), 궤적은 압축 가능

그림 2-4 에이전트가 모델을 호출할 때마다 구성되는 컨텍스트

교재 52pg 中



What survives compaction

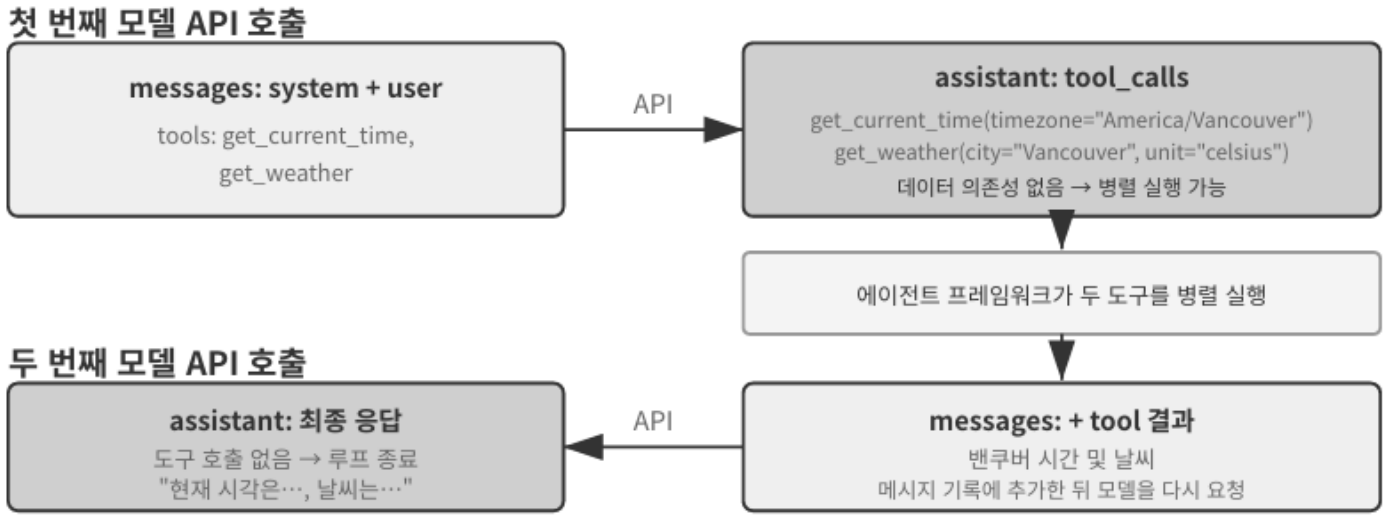
When a long session compacts, Claude Code summarizes the conversation history to fit the context window. As of v2.1.198, the summarization request inherits your session's **extended thinking** configuration, so it reasons with thinking enabled when your session has it enabled and stays off otherwise. Thinking affects only how the summary is produced; your session settings are unchanged afterward. What happens to each kind of content depends on how it was loaded:

Mechanism	After compaction
System prompt and output style	Both still apply
Project-root CLAUDE.md and unscoped rules	Re-injected from disk
Auto memory	Re-injected from disk
Git status snapshot	Claude Code reads a fresh one from your repository
The plan Claude wrote in **plan mode**	Re-injected from disk
Rules with `paths:` frontmatter	Claude Code reloads them **on demand**
Nested CLAUDE.md in subdirectories	Claude Code reloads them **on demand**
Files Claude read or edited	Claude Code re-reads up to five, most recently modified first
Invoked skill bodies	Re-injected, capped at 5,000 tokens per skill and 25,000 tokens total; oldest dropped first
Background commands and background **subagents**	Keep running. Claude Code reminds Claude which ones are still running so it doesn't start a duplicate
Context that hooks added earlier	Summarized with the rest of the conversation
SessionStart hooks that match the `compact` source	Claude Code runs them and adds their output to the compacted context

- (좌) Claude Code에서의 Compact 기능은 뒤쪽 궤적을 요약함.
- (우명균) Claude Code Docs에서의 Compaction 설명.

3-2. Context Engineering(1): 뒷 쪽 궤적 흐름

- 아래 그림의 '두 번의 API 호출' 흐름을 보면, 사용자의 질문을 받은 모델이 도구 호출을 요청하고, 프레임워크가 도구를 실행한 뒤 그 결과를 대화 기록에 추가해 모델을 다시 호출함
- 이 흐름이 1주차 코드과제에서 만든 chatbot ⇆ tools 루프의 구조임! 간단하게 언급만 하고 넘어가겠음!!



무상태 API에서는 매 라운드마다 전체 메시지 기록을 모델에 다시 전송해야 합니다.

그림 2-3 두 번의 모델 API 호출에 대한 전체 상호작용 과정

교재 45pg 中

3-2. Context Engineering(2): Messages와 Chat Completions API

- 좌측 하단의 교재 43pg의 내용은 2주차 코드과제에서 제시됨. LangChain과 LangGraph의 메시지 클래스는 이런 API의 메시지 구조를 파이썬 객체로 다루도록 해줌.
- Chat Completions API는 좌측 하단의 구조화된 API 메시지를 오른쪽의 Chat Template으로 바꿔줌.
- 우리(에이전트 개발자)는Chat Template을 직접 작성하거나 수정할 필요가 없음!! API 서버가 자동으로 처리함. 다만 기억해야할 것은 표준 API 형식을 사용해야 한다는 각오 ㅎㅎ

2.2 에이전트는 대규모 모델을 어떻게 호출하는가: API의 컨텍스트 구조 이해

이 절에서는 OpenAI의 Chat Completions API를 구체적인 예로 사용합니다. Anthropic, Google 등 다른 공급자는 세부 사항이 다르지만, 에이전트용 API는 구조화된 대화 기록과 사용 가능한 도구 정의를 조합해 각 모델 호출을 구성한다는 공통된 패턴을 따릅니다. 이 구조를 이해하는 일은 이 장에서 뒤이어 설명할 컨텍스트 엔지니어링 기법의 토대입니다.

2.2.1 메시지의 네 가지 역할

Chat Completions 계열 API의 핵심 입력은 보통 `messages`라고 부르는 **메시지 목록**입니다. 각 메시지의 `role` 필드는 모델이 메시지를 어떻게 해석해야 하며 어디에서 왔는지를 알려 줍니다.

- **system**: 에이전트의 정체성, 행동, 제약, 작업 흐름을 정의하는 개발자 작성 지시입니다. 모델은 이를 우선순위가 높은 지시로 취급합니다. 대부분의 대화에서 시스템 메시지는 메시지 목록 맨 앞에 한 번 나타납니다.
- **user**: 최종 사용자의 입력으로, 에이전트가 처리해야 할 요청입니다.
- **assistant**: 자연어 응답과 도구 호출 요청을 포함한 이전 모델 출력입니다. 여러 라운드의 상호작용에서는 이후 요청에도 이 메시지를 포함하여, 상태를 갖지 않는 다음 모델 호출이 이전 궤적에 접근하게 합니다.
- **tool**: 에이전트 프레임워크가 도구를 실행한 뒤 반환한 결과입니다. 각 도구 결과는 `tool_call_id`를 통해 해당 도구 호출에 연결되므로 모델은 어느 결과가 어느 요청에서 나왔는지 알 수 있습니다.

도구 정의는 메시지가 아닙니다. 별도의 `tools` 필드로 제공하며, 모델이 사용할 수 있는 도구와 각 도구가 받는 파라미터를 선언합니다.

교재 43pg 中

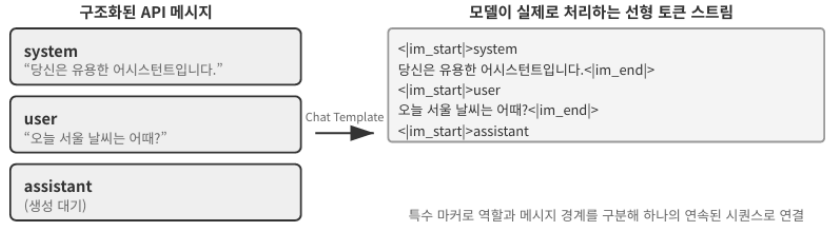


그림 2-8 Chat Template의 토큰 구조

Chat Template을 **봉투 형식**이라고 생각하면 이해하기 쉽습니다. API 메시지가 편지의 내용이라면 Chat Template은 봉투에 보내는 사람, 받는 사람, 경계를 어떻게 적을지 정합니다. `<|im_start|>system`, `<|im_end|>` 같은 특수 토큰으로 각 메시지의 역할과 경계를 표시합니다. Qwen, Llama, Gemma 등 모델 계열마다 서로 다른 봉투 형식을 사용합니다. vLLM, Ollama 같은 API 서버가 모델의 Chat Template에 맞춰 이 변환을 자동으로 수행하므로 개발자가 직접 다룰 필요는 없습니다.

Qwen 모델 계열을 예로 들면 같은 대화가 API 수준과 모델 내부에서 전혀 다른 형태로 나타납니다.

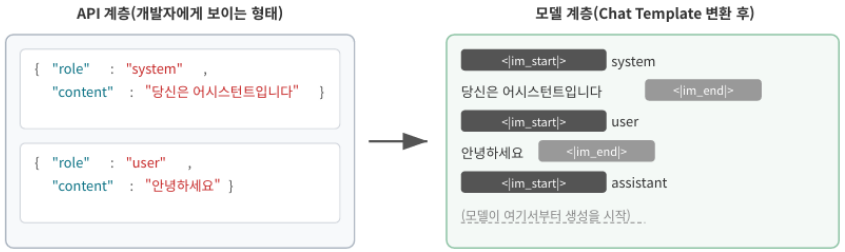


그림 2-9 API 메시지에서 모델 토큰 스트림으로의 변환

교재 59pg 中

3-2. Context Engineering(3): KV Cache가 무엇이냐! (이것만은 기억하자)

- 모델은 토큰을 하나 생성할 때마다 앞선 토큰들의 중간 계산 결과를 참조함. 매번 이를 처음부터 계산하면 컨텍스트가 길어질수록 계산 비용이 커짐.
- KV Cache는 중간 Key·Value 상태를 저장해 이후 계산에서 재사용함.
- 재사용할 컨텍스트의 토큰 접두부는 변하지 않아야 함. 토큰 시퀀스가 달라지면 처음 달라진 토큰부터 이후의 KV 상태를 다시 계산함. 그보다 앞선 KV 상태는 변경의 영향을 받지 않음.

이 직관을 염두에 두고 프로덕션 장애 사례를 살펴보겠습니다. 어느 팀의 고객 서비스 에이전트는 하루 10만 건의 대화를 처리하며 정상적으로 작동하고 있었습니다. 한 엔지니어가 현재 시각을 에이전트에 알려 주려고 시스템 프롬프트에 `Current time: {{now}}`라는 한 줄을 추가해 타임스탬프를 실시간으로 주입했습니다. 다음 날 모니터링 경보가 울렸습니다. 모든 대화의 TTFT가 0.5초에서 3~5초로 늘었고, 월간 추론 비용은 거의 두 배가 됐습니다. 코드는 올바르게 보였고 모델도 바뀌지 않았습니다. 문제는 컨텍스트에 있었습니다.

타임스탬프 한 줄 때문에 요청할 때마다 토큰 시퀀스가 타임스탬프 위치부터 달라져, 그 위치와 이후의 KV 상태를 재사용할 수 없었습니다. 시스템 프롬프트가 컨텍스트 앞부분에 있으므로 모델은 그 뒤에 오는 입력 토큰 대부분의 Key-Value 쌍을 다시 계산해야 하는 경우가 많았습니다. 여기서 Key와 Value는 어텐션 메커니즘에 쓰이는 두 종류의 벡터이며, 아래 실험 2-2에서 그 역할을 시각적으로 보여 줍니다. 이처럼 눈에 보이지 않는 비용은 에이전트 시스템에서 되풀이됩니다. 무해해 보이는 코드 한 줄이 전체 추론 파이프라인을 10배 가까이 느리게 만들 수 있습니다. 이 절에서는 이러한 함정을 피하는 방법을 설명합니다.

> **기술 참고**: 이 절은 Transformer의 어텐션 메커니즘과 KV Cache의 내부 원리를 다루므로 이 책에서 기술적 밀도가 가장 높은 부분 중 하나입니다. 이러한 기반 메커니즘이 익숙하지 않다면 **세부 원리는 건너뛰고 다음 세 가지 핵심 결론만 기억해도 됩니다.**
>
> 1. **시스템 프롬프트와 도구 정의를 확정한 뒤에는 바꾸지 마세요.** 공백 하나를 추가하는 사소한 수정도 토큰 시퀀스를 바꿔 처음 다른 토큰부터 이후의 캐시를 재사용하지 못하게 할 수 있습니다. 변경 위치가 앞쪽일수록 일반적으로 지연 시간과 비용에 미치는 영향이 큽니다. 정확한 정도는 모델과 설정에 따라 달라집니다.
> 2. **동적 정보는 항상 끝에 추가하세요.** 타임스탬프나 사용자 상태처럼 변하는 콘텐츠는 기존 시스템 프롬프트를 수정하지 말고 대화 끝에 새 메시지로 추가해야 합니다.
> 3. **표준 API 형식을 사용하고 메시지를 직접 이어 붙이지 마세요.** 구조화된 메시지는 Chat Template을 거쳐 모델이 학습할 때 접한 고정 토큰 시퀀스로 변환됩니다. `"USER: ... ASSISTANT: ..."`처럼 문자열을 직접 이어 붙이는 방식의 근본적인 문제는 이 학습 형식에서 벗어나 모델의 다단계 사고 능력을 약화한다는 점입니다. 다만 캐시는 결과 토큰 시퀀스에만 의존합니다. 직접 이어 붙인 접두부도 바이트 단위로 안정적이면 캐시할 수 있습니다. 캐시는 그 접두부가 달라질 때만 무효화됩니다. 예를 들면 접두부 안에 동적 콘텐츠를 삽입한 경우입니다.
>
> 세 결론의 직관은 간단합니다. 대형 모델은 컨텍스트를 처리할 때 이미 처리한 내용을 캐시하므로, 다음번에는 새로 추가된 부분만 처리하면 됩니다.
>
> 이 세 원칙만 기억해도 아래의 기술적인 세부 내용을 건너뛰면서 에이전트의 컨텍스트 구조를 올바르게 설계할 수 있습니다. 다음 내용은 "왜 그런가"를 더 깊이 이해하려는 독자를 위한 것입니다.

교재 54pg

3-2. Context Engineering(4): Messages와 Chat Completions API

- Production 수준 제품들의 시스템 프롬프트는 이 캐시 경계에 맞춰짐. 변동될 수 있는 부분들은 최대한 뒤에 들어가게 됨.
- 상위 에이전트와 하위 에이전트를 구성할 때, 하위 에이전트가 상위 에이전트의 컨텍스트를 상속한다면 하위 에이전트의 프롬프트, 도구 정의, 모델 설정, 메시지 접두 부, 사고 설정은 상위 에이전트와 바이트 단위로 일치해야함. → api 공급자의 Prompt Cache를 사용할 수 있다는 장점이 생김.

[피해야할 것!!]

- 라운드 시스템 프롬프트를 다시 만드는 것(현재 시각이나 모드 스위치를 써넣는 등),
- 첫 사용자 메시지 안의 환경 정보를 매 라운드 다시 렌더링하는 것,
- 도중에 tools를 추가하거나 제거하는 것, 오래된 도구 결과를 제자리에서 잘라내는 것,
- 오래된 메시지에 알림을 끼워 넣었다가 다시 지우는 것

[꺼드럭댈 것!!]
https://platform.claude.com/docs/en/build-with-claude/preserved-thinking

3-2. Context Engineering(5): Context 설계

- 여기서 말하는 컨텍스트는 '시스템 프롬프트'임.

프롬프트 엔지니어링 관련

- 어조 스타일 관련: NEVER do X"처럼 대문자를 사용한 표현은 "Please avoid doing X"같은 부드러운 표현보다 지시를 더 두드러지게 하지만, 남용하면 효과가 약해짐
- 형식 관련: 학습 데이터에 구조화된 콘텐츠가 많기 때문입니다. XML 태그는 계층 원칙을 따르며 태그 이름 자체가 의미를 전달함. <working_directory>는 작업 디렉터리 정보임을 즉시 알려 주지만,
- "Current directory: /Users/project/src"같은 일반 텍스트 형식에서는 콜론 양쪽의 관계를 추론하는 추가 과정이 필요함.
- 가장 좋은 것은 Markdown + XML 조합

```
# 도구 사용 규칙

## 파일 작업
<file_operation>
- 파일을 읽기 전에 경로가 존재하는지 먼저 확인한다
- 파일에 쓰기 전에 먼저 백업한다
</file_operation>

## 네트워크 요청
<network_request>
- 타임아웃은 30초로 설정한다
- 실패하면 최대 3회까지 재시도한다
</network_request>
```

- **Markdown의 역할**: #, ## 같은 제목은 사람이 계층 구조를 한눈에 파악하게 해 주어 가독성이 좋습니다.
- **XML의 역할**: <file_operation>, <network_request> 같은 태그는 "이 블록은 파일 작업에 관한 것", "이 블록은 네트워크 요청에 관한 것"이라고 모델에 알려 줍니다. 의미가 정확하므로 모델도 더 정확하게 처리합니다.

교재 57pg 中

+. 다 아는 내용

- 사람의 인지 부하를 줄이는 방법은 대규모 언어 모델에도 똑같이 효과적임.
- 모델이 학습 과정에서 인간의 언어와 사고 패턴을 배웠기 때문
- 퓨샷. 주면 당연히 좋고

3-2. Context Engineering(6): Skills

- 에이전트가 더 많은 시나리오를 처리하게 되면 시스템 프롬프트는 계속 늘어나는 경향이 있음
- → 토큰낭비와 컨텍스트 부패(컨텍스트에 무관한 정보가 너무 많으면 핵심 콘텐츠에 대한 모델의 주의가 분산되는 현상)

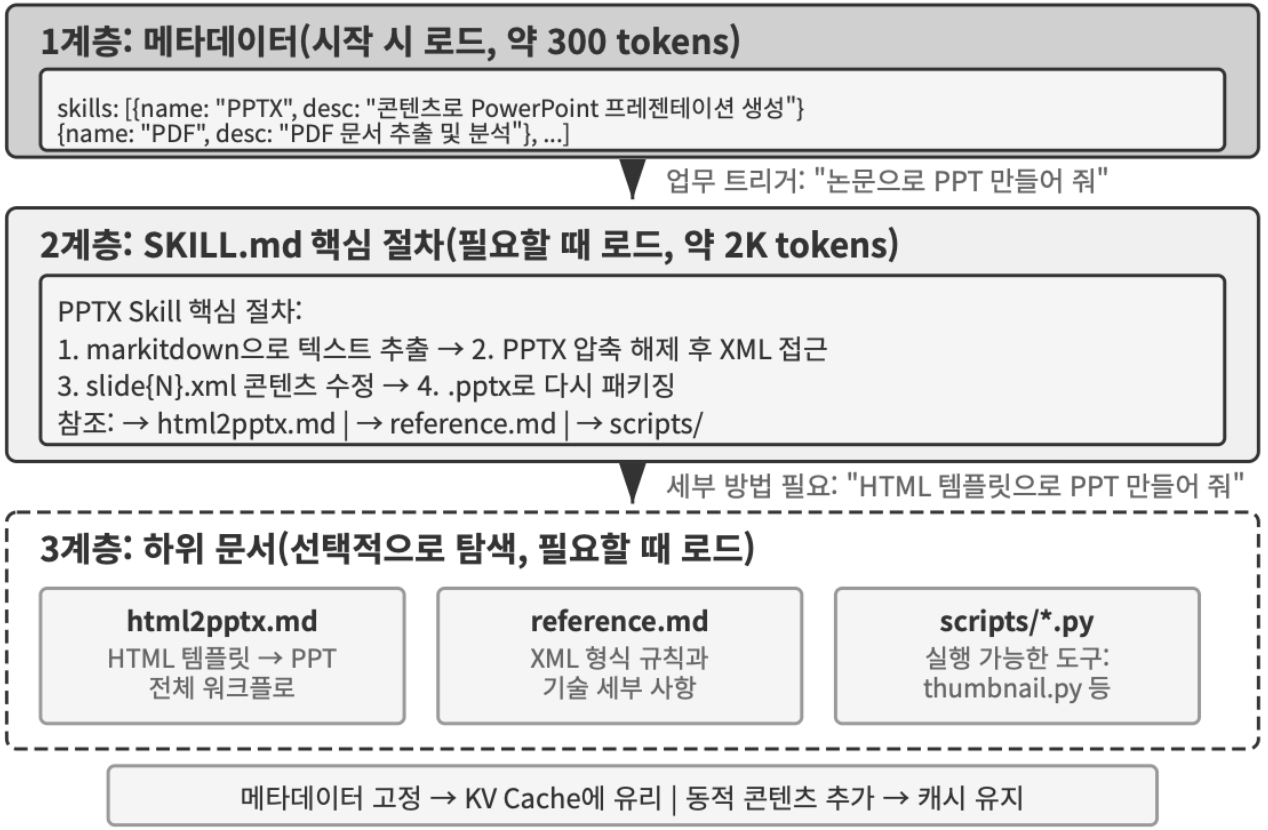


그림 2-11 스킬의 점진적 공개 메커니즘

교재 62pg 中

3-2. Context Engineering(6): Skills

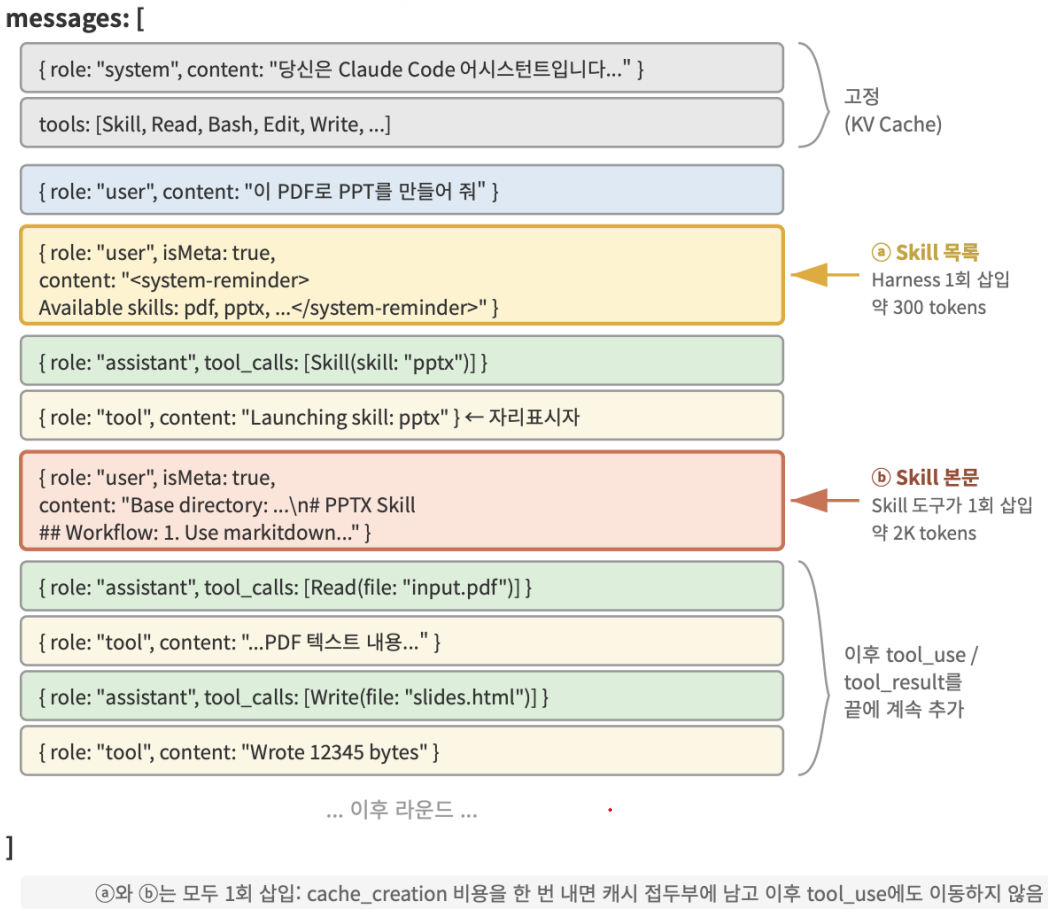


그림 2-12 스킬을 활성화한 에이전트 궤적의 전체 구조

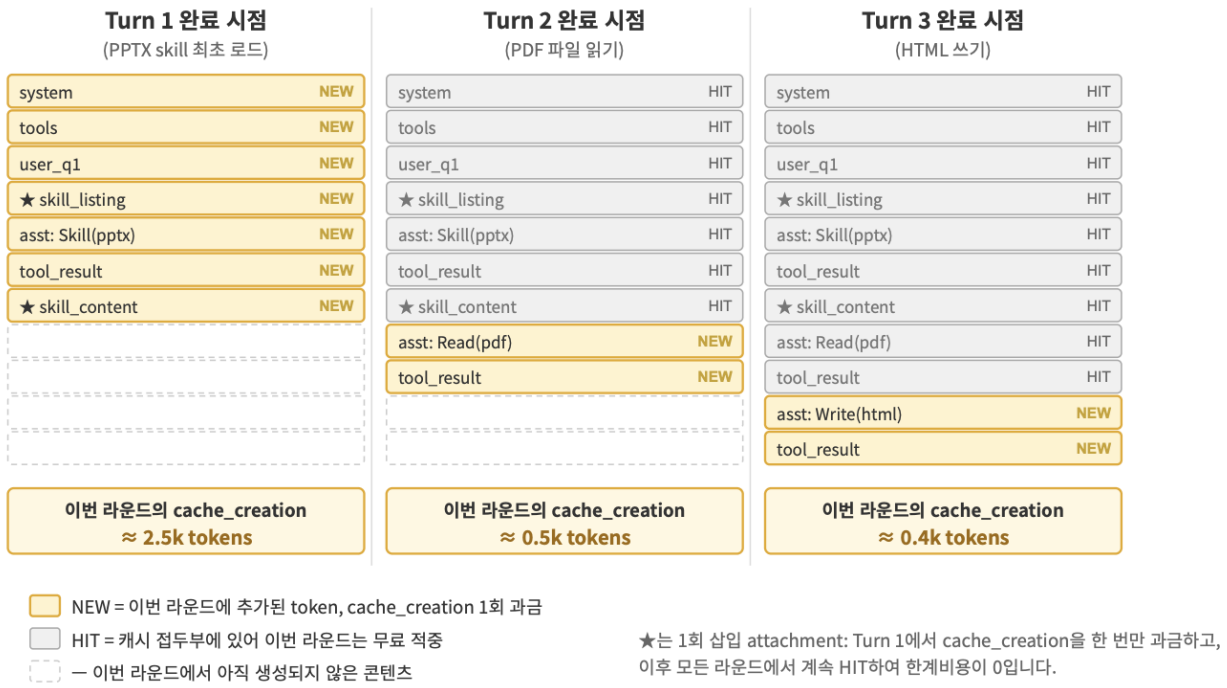


그림 2-13 에이전트 궤적이 늘어날 때 KV Cache의 변화

교재 65pg 中

3-2. Context Engineering(7): Skills VS Tool

2.5.4 스킬과 도구의 관계

컨텍스트 관리의 관점에서 스킬 메커니즘은 KV Cache에 매우 친화적입니다. 전문 코드 도구의 정의를 모두 시스템 프롬프트에 넣으면 도구 수가 늘수록 많은 토큰을 소비하고 모델의 주의를 방해합니다. 반면 스킬 + 범용 실행기 방식에서는 도구 수를 적게 유지할 수 있습니다(5장에서 보듯 핵심 도구 7개만 필요합니다). 스킬 내용은 앞서 설명한 점진적 공개 메커니즘으로 필요할 때 불러오므로 캐시된 접두부에 영향을 주지 않습니다. 4장에서는 두 형태를 자세히 비교하고 선택하는 틀을 제시하며, 9장에서는

교재 66pg 中